SQL JOINS

Corrected Study Guide for RAG Retrieval

Contents

1. 1. Introduction to SQL Joins
2. 2. Sample Tables
3. 3. INNER JOIN
4. 4. LEFT JOIN
5. 5. RIGHT JOIN
6. 6. FULL OUTER JOIN
7. 7. CROSS JOIN and Self Join
8. 8. Join Conditions and NULL
9. 9. Comparison at a Glance
10. 10. Common Mistakes
11. 11. Interview Practice

Purpose: a structured study guide designed for topic-wise learning and retrieval practice.

1. Introduction to SQL Joins

A SQL join combines rows from two tables using a related column or a join condition. Joins let you query normalized data without storing the same fact repeatedly.

```
SELECT s.student_id, s.name, e.course_id
FROM Students AS s
JOIN Enrollments AS e
  ON s.student_id = e.student_id;
```

Use explicit JOIN ... ON syntax. It makes the relationship visible and avoids the accidental Cartesian products that old comma-join syntax can produce.

2. Sample Tables

```
Students
student_id | name
1          | Asha
2          | Ravi
3          | Meera

Enrollments
student_id | course
1          | SQL
2          | Python
4          | Java
```

The sample deliberately includes student 3 without an enrollment and enrollment 4 without a matching student. Those unmatched rows make outer-join behavior easier to see.

3. INNER JOIN

INNER JOIN returns only rows that satisfy the join condition in both tables.

```
SELECT s.student_id, s.name, e.course
FROM Students AS s
INNER JOIN Enrollments AS e
  ON s.student_id = e.student_id;
```

Expected matches: Asha with SQL and Ravi with Python. Meera and the enrollment for student 4 are excluded.

4. LEFT JOIN

LEFT JOIN returns every row from the left table and matching rows from the right table. If there is no match, right-side columns are NULL.

```
SELECT s.student_id, s.name, e.course
FROM Students AS s
LEFT JOIN Enrollments AS e
  ON s.student_id = e.student_id;
```

Meera remains in the output with NULL for course. A frequent mistake is putting a right-table filter in WHERE when unmatched left rows should remain; that can turn the result into inner-join-like behavior.

5. RIGHT JOIN

RIGHT JOIN returns every row from the right table and matching rows from the left table. Unmatched left-side columns are NULL.

```
SELECT s.student_id, s.name, e.course
FROM Students AS s
RIGHT JOIN Enrollments AS e
  ON s.student_id = e.student_id;
```

The enrollment for student 4 remains, with NULL student details. Many teams prefer rewriting RIGHT JOIN as LEFT JOIN by swapping table order for readability.

6. FULL OUTER JOIN

FULL OUTER JOIN returns matched rows plus unmatched rows from both sides. Unmatched columns are filled with NULL.

MySQL does not support FULL OUTER JOIN as a direct keyword. A common MySQL pattern combines a LEFT JOIN with a RIGHT-only set using UNION ALL.

```
SELECT s.student_id, s.name, e.course
FROM Students AS s
LEFT JOIN Enrollments AS e
  ON s.student_id = e.student_id
UNION ALL
SELECT s.student_id, s.name, e.course
FROM Students AS s
RIGHT JOIN Enrollments AS e
  ON s.student_id = e.student_id
WHERE s.student_id IS NULL;
```

The second branch keeps only right-side rows without a left-side match. UNION ALL avoids unnecessary duplicate elimination; the anti-match condition prevents matched rows from appearing twice.

7. CROSS JOIN and Self Join

CROSS JOIN

CROSS JOIN returns every combination of rows. If the left table has m rows and the right table has n rows, the result has m × n rows.

```
SELECT s.name, c.course_name
FROM Students AS s
CROSS JOIN Courses AS c;
```

Self join

A self join joins a table to itself, often to represent hierarchies such as employees and managers.

```
SELECT e.name AS employee, m.name AS manager
FROM Employees AS e
LEFT JOIN Employees AS m
  ON e.manager_id = m.employee_id;
```

8. Join Conditions and NULL

• Use ON to state how rows relate; use WHERE to filter the joined result.

• NULL does not equal NULL under ordinary equality comparison. Use IS NULL or IS NOT NULL.

• One-to-many joins can multiply rows. Check the relationship cardinality before interpreting counts.

• Use qualified names such as s.student_id when multiple tables share column names.

• For an outer join, a filter on the optional side in WHERE may remove rows with NULLs. Consider putting that filter in ON when preserving unmatched rows is required.

9. Comparison at a Glance

Join	Rows returned
INNER	Only matching rows
LEFT	All left rows, plus right matches
RIGHT	All right rows, plus left matches
FULL OUTER	All rows from both sides, matched where possible
CROSS	Every left/right combination
SELF	A table joined to itself; type depends on chosen join

10. Common Mistakes

• Forgetting the ON condition and creating a huge Cartesian result.

• Joining on a non-unique column and unexpectedly multiplying rows.

• Using = NULL instead of IS NULL.

• Filtering right-side columns in WHERE and unintentionally losing LEFT JOIN unmatched rows.

• Assuming MySQL accepts FULL OUTER JOIN directly.

• Using UNION instead of UNION ALL without considering duplicate-removal cost and semantics.

11. Interview Practice

• Return every student, including those without an enrollment. Which join is suitable?

• Return enrollments even when the student record is missing. Which join is suitable?

• Explain the difference between ON and WHERE for an outer join.

• Write a MySQL query that emulates FULL OUTER JOIN.

• What is the maximum number of rows produced by a CROSS JOIN of 7 and 4 rows?

• Why might a join return more rows than the left table?